PT-P3: Implementing Simple and Multiple Linear Regression

Leonardo Antero SP. Yoro
IT Elective III, BSIT
Jose Rizal University
Mandaluyong, Philippines
leonardoantero.yoro@my.jru.edu

***Abstract*— This study used simple and multiple linear regression analysis to identify a relationship between the performance metrics of professional basketball players and field goal attempts per game. Utilizing the 2022-2023 NBA Player Stats dataset, the researcher was able to identify through correlation which variables most directly predict FGA: PTS, FG, and 2PA. The simple regression model with PTS as the only predictor had an R^2 score of 0.969, and the multiple regression model with additional features slightly improved the accuracy. The results emphasize the significance of feature selection in predictive modeling and reinforce the effectiveness of regression analysis in sports analytics.**

Keywords*— *Simple Linear Regression, Multiple Linear Regression, NBA Statistics, Predictive Modeling, Attribute Selection.

I. Introduction

Predictive modeling is an important technique in the era of data-driven decision-making to analyze and forecast real-world phenomena. One of the fundamental techniques in predictive analytics is linear regression, which establishes relationships between dependent and independent variables to make accurate predictions. This exercise aims to enhance our understanding of data collection, attribute selection, and predictive modeling using simple and multiple linear regression.

The study is based on the 2022-2023 NBA Player Stats dataset [1], sourced from Kaggle, comprising different performance metrics of professional basketball players. The best single predictor for the target variable is found to be determined using simple linear regression, and through multiple linear regression, we arrive at a more effective model with multiple relevant attributes.

To ensure meaningful predictor variables are selected, I applied feature selection for regression analysis. Attribute selection is a critical component of regression modeling as it eliminates irrelevant or redundant variables, which enhances model performance and interpretability. I applied correlation analysis, one of the most common filter-based feature selection techniques, to measure the strength of relationships between variables [2]. This means I can reduce dimensionality with attributes with the highest correlation to the target variable for use in the model while ensuring the most impactful features for a good predictive modeling. The outcome is the enhanced reliability and accuracy of our regression models in eliminating overfitting, where all selected attributes can make a meaningful contribution to predictions.

II. Data Collection

1. *Dataset Description*

The study uses the 2022-2023 NBA Player Stats dataset [1], which contains performance metrics for professional basketball players during the 2022-2023 official NBA season. The dataset was found in kaggle, a well-known area in the Internet that contains vast information of datasets, covering almost all industries in the world. This dataset is of particular interest to the study because it offers a wide dimension of numerical attributes that could be. The dataset has a shape of (679,30), stating that there are 679 existing records and 30 attributes.

```
print("Initial shape of the dataset:", df.shape)
Initial shape of the dataset: (679, 30)
```

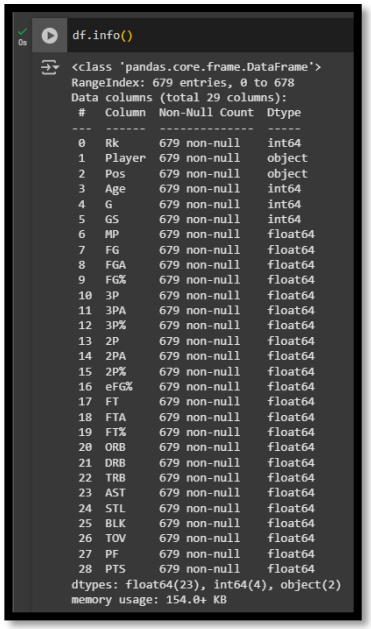
Fig. 1. Initial Shape of the Dataset

```
[ ] print("Intial columns of the dataset:\n", df.columns)
Intial columns of the dataset:
 Index(['Rk', 'Player', 'Pos', 'Age', 'Tm', 'G', 'GS', 'MP', 'FG', 'FGA', 'FG%',
       '3P', '3PA', '3P%', '2P', '2PA', '2P%', 'eFG%', 'FT', 'FTA', 'FT%',
       'ORB', 'DRB', 'TRB', 'AST', 'STL', 'BLK', 'TOV', 'PF', 'PTS'],
      dtype='object')
```

Fig. 2. Initial Columns of the Dataset

df.info() was also executed, in order to display the number of records, column names, its data types, memory usage, and non-null counts. The command helps the researchers to identify missing values and analyze the dataset structure. Df.dtypes() was also executed to display the data types of each attributes in the DataFrame. This further helps in data preprocessing and analysis as it determines whether the attribute contains numerical or categorical data.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE



```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 679 entries, 0 to 678
Data columns (total 29 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Rk      679 non-null    int64
 1   Player  679 non-null    object
 2   Pos     679 non-null    object
 3   Age     679 non-null    int64
 4   G       679 non-null    int64
 5   GS      679 non-null    int64
 6   MP      679 non-null    float64
 7   FG      679 non-null    float64
 8   FGA     679 non-null    float64
 9   FG%     679 non-null    float64
 10  3P      679 non-null    float64
 11  3PA     679 non-null    float64
 12  3P%     679 non-null    float64
 13  2P      679 non-null    float64
 14  2PA     679 non-null    float64
 15  2P%     679 non-null    float64
 16  eFG%    679 non-null    float64
 17  FT      679 non-null    float64
 18  FTA     679 non-null    float64
 19  FT%     679 non-null    float64
 20  ORB     679 non-null    float64
 21  DRB     679 non-null    float64
 22  TRB     679 non-null    float64
 23  AST     679 non-null    float64
 24  STL     679 non-null    float64
 25  BLK     679 non-null    float64
 26  TOV     679 non-null    float64
 27  PF      679 non-null    float64
 28  PTS     679 non-null    float64
dtypes: float64(23), int64(4), object(2)
memory usage: 154.0+ KB
```

Fig. 3. df.info() command execution

2. *Dataset Attributes*

The dataset [1] consists of 679 columns with 30 attributes, and the description along with the relevance of the attributes are as follows:

a) Rk (Rank): Rank is acting as an index of the dataset

b) Player: Identifies the player. It is essential for tracking individual statistics.

c) Pos (Position): Determines the role a player plays on the court, which influences their responsibilities and statistical expectations.

d) Age: Helps analyze the development of players, peak years of performance, and the career span.

e) Tm (Team): It identifies the team, through which team-based performance analysis is possible.

f) G (Games Played): It indicates how available a player is for the season.

g) GS (Games Started): Which states the player's role for his team as he may start a game or play from the bench.

h) MP (Minutes per game): The value indicates up to what extent a player can play in a game and therefore affects the entire statistics.

i) FG (Field Goals per Game): It measures scoring ability by indicating how many field goals he makes.

j) FGA (Field Goal Attempts per Game): Reflects shooting frequency, thereby evaluating shot selection.

k) FG% (Field Goal Percentage): Evaluates the efficiency of scoring from the field.

l) 3P (3-Point Field Goals per Game): Tracks the contribution of long-range shooting.

m) 3PA (3-Point Attempts per Game): Shows how often a player takes three-pointers.

n) 3P% (3-Point Percentage): Measures the accuracy of three-point shooting.

o) 2P (2-Point Field Goals per Game): Shows scoring from inside the arc.

p) 2PA (2-Point Attempts per Game): Reflects the number of times a player attempts a two-point shot

q) 2P% (2-Point Percentage): A measure of field goal percentage as it applies to two-point shots

r) eFG% (Effective Field Goal Percentage): Measures shooting efficiency with the extra weight a team gets from three-pointers

s) FT (Free Throws per Game): Converting from free-throw opportunities into points scored

t) FTA (Free Throw Attempts per Game): Reflects how many times a player is awarded free-throws.

u) FT% (Free Throw Percentage): Reveals the conversion efficiency of free throws.

v) ORB (Offensive Rebounds per Game): Measures how well a player can grab offensive rebounds, turning into second-chance points.

w) DRB (Defensive Rebounds per Game): Tracks defensive rebounding ability that is crucial for stopping opponent possessions.

x) TRB (Total Rebounds per Game): Measures a player's overall contribution to rebounding.

y) AST (Assists per Game): Reflects playmaking by measuring how well a player is able to get teammates into scoring positions.

z) STL (Steals per Game): Represents the ability to defend by taking the ball away from opponents.

aa) BLK (Blocks per Game): Measures the number of times a player protects the rim and contests shots.

bb) TOV (Turnovers per Game): Tracks the mistakes made in passing and ball handling, which could hurt the team.

cc) PF (Personal Fouls per Game): Reflects the aggressive defense; more fouls committed may result in penalties.

dd) PTS (Points per Game): Measures scoring contribution, a critical factor for evaluating a player's offensive impact.

3. *Data Cleaning and Data Preprocessing*

The data cleaning and data preprocessing was conducted in Google Collaboratory, inside an .ipynb Jupyter Notebook file that was provided to the researcher.

a) Data preprocessing for further data cleaning

The Tm (team) attribute was removed as it does not bring relevance to the objective of the study. It was then removed using this command:

```
columns_to_drop = ["Tm"]
df = df.drop(columns=columns_to_drop)
```

```
[ ] df.head()
```

	Rk	Player	Pos	Age	G	GS	MP	FG	FGA	FG%	...	FT%	ORB	DRB	TRB	AST	STL	BLK	TOV	PF
0	1	Precious Achiuwa	C	23	55	12	20.7	3.6	7.3	0.485	...	0.702	1.8	4.1	6.0	0.9	0.6	0.5	1.1	1.9
1	2	Steven Adams	C	29	42	42	27.0	3.7	6.3	0.597	...	0.364	5.1	6.5	11.5	2.3	0.9	1.1	1.9	2.3
2	3	Bam Adebayo	C	25	75	75	34.6	8.0	14.9	0.540	...	0.806	2.5	6.7	9.2	3.2	1.2	0.8	2.5	2.8
3	4	Ochai Agbaji	SG	22	59	22	20.5	2.8	6.5	0.427	...	0.812	0.7	1.3	2.1	1.1	0.3	0.3	0.7	1.7
4	5	Santi Aldama	PF	22	77	20	21.8	3.2	6.8	0.470	...	0.750	1.1	3.7	4.8	1.3	0.6	0.6	0.8	1.9

5 rows × 29 columns

Fig. 4. Removal of the Tm attribute and the display of records after removal.

The command *df.hist(bins=50, figsize=(20,15))* generates histograms for all numerical attributes in the DataFrame. Each histogram plots the distribution of values in a column so that one may easily find the trends, skewness, or outliers. Here, *bins=50* sets the number of bins for grouping the values, *figsize=(20,15)* scales the figure for better readability. Finally, *plt.show()* draws the plots.

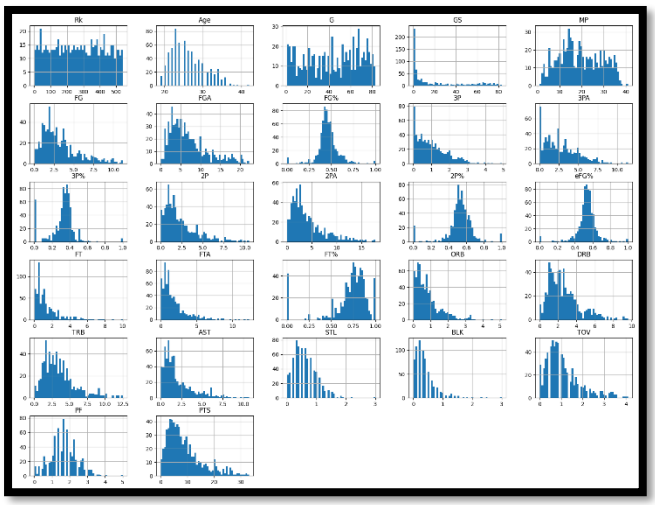


Fig. 5. Histogram display of the current, uncleaned attribtues

The following loop goes over each column of the DataFrame, providing deep insights in the dataset about the number of unique values and missing values within each attribute. The unique value count will inform us if the column is categorical or numerical data and evaluate how much variability is present in a numerical feature. Meanwhile, it will also highlight missing values and allow us to understand possible quality issues with our data and which handling technique will be suitable such as imputation or removal.

```
for column in df:
    print(f"Unique values in {column}:", df[column].nunique())
    print(f"Missing values in {column}:", df[column].isnull().sum())
    print()

Unique values in 3P: 40
Missing values in 3P: 0

Unique values in 3PA: 92
Missing values in 3PA: 0

Unique values in 3P%: 214
Missing values in 3P%: 0

Unique values in 2P: 83
Missing values in 2P: 0

Unique values in 2PA: 134
Missing values in 2PA: 0

Unique values in 2P%: 273
Missing values in 2P%: 0

Unique values in eFG%: 275
Missing values in eFG%: 0

Unique values in FT: 67
Missing values in FT: 0

Unique values in FTA: 80
Missing values in FTA: 0

Unique values in FT%: 263
Missing values in FT%: 0

Unique values in ORB: 39
Missing values in ORB: 0

Unique values in DRB: 79
Missing values in DRB: 0

Unique values in TRB: 101
Missing values in TRB: 0
```

Fig. 6. Output of code for finding unique and missing values for each attribute

b) Identifying Missing Values

According to Nasima (2024) [3], dealing with missing values is valuable in creating an accurate and reliable prediction model, as successful handling of missing values will lead to optimal data analysis and machine learning models.

This code calculates the number of missing values in each column of the dataset and prints the results as a dictionary. It helps to identify data gaps that may need imputation or removal before further analysis.

```
missing_values = df.isnull().sum()
print(missing_values.to_dict())

0, 'GS': 0, 'MP': 0, 'FG': 0, 'FGA': 0, 'FG%': 0, '3P': 0, '3PA': 0, '3P%': 0, '2P': 0, '2PA': 0, '2P%
```

Fig. 7. Output of code for identifying missing values for each attribute

```
[20] print("Columns of the dataset after handling missing values:\n", df.columns)

     print("\nShape of DataFrame after removing columns with missing values:", df.shape)

Columns of the dataset after handling missing values:
 Index(['Rk', 'Player', 'Pos', 'Age', 'G', 'GS', 'MP', 'FG', 'FGA', 'FG%', '3P',
        '3PA', '3P%', '2P', '2PA', '2P%', 'eFG%', 'FT', 'FTA', 'FT%', 'ORB',
        'DRB', 'TRB', 'AST', 'STL', 'BLK', 'TOV', 'PF', 'PTS'],
       dtype='object')

Shape of DataFrame after removing columns with missing values: (679, 29)
```

Fig. 8. Shape of the DataFrame after handling missing values

It has concluded that there were no missing values, upon executing various commands that helps the study get close to optimal data analysis, thus, moving forward to handling duplicate values and records. Finally, the figure above (Fig. 8) displays the current shape of the DataFrame after handling the missing values.

c) Identifying Duplicate Values

Nick End from row zero (2023) [4] explains that duplicate values can arise due to errors in data entry, system malfunctions, or data merging from several sources. Identification and removal of duplicate data are therefore of great importance for preserving data integrity and avoiding overrepresentation of certain observations. In the current dataset used and industry it was from, duplicate values may appear because of players becoming too similar and hence produce similar statistics.

The following code prints the number of unique values in each column within the dataset and includes the count of duplicates for each column, which gives insight into variability within the data and may suggest redundancy that should be cleaned and preprocessed

```
for column in df.columns:
    print(f"Unique values in {column}:", df[column].nunique())
    print(f"Number of duplicates in '{column}' column: {df[column].duplicated().sum()} \n")
```

Fig. 9. Lines of code used for identifying unique and duplicate values for each attribute

```
Unique values in Rk: 539
Number of duplicates in 'Rk' column: 140

Unique values in Player: 539
Number of duplicates in 'Player' column: 140

Unique values in Pos: 8
Number of duplicates in 'Pos' column: 671

Unique values in Age: 22
Number of duplicates in 'Age' column: 657

Unique values in G: 83
Number of duplicates in 'G' column: 596

Unique values in GS: 80
Number of duplicates in 'GS' column: 599

Unique values in MP: 308
Number of duplicates in 'MP' column: 371

Unique values in FG: 101
Number of duplicates in 'FG' column: 578

Unique values in FGA: 174
Number of duplicates in 'FGA' column: 505

Unique values in FG%: 279
Number of duplicates in 'FG%' column: 400

Unique values in 3P: 40
Number of duplicates in '3P' column: 639
```

Fig. 10. Results of Fig. 9. code

```
Unique values in 3PA: 92
Number of duplicates in '3PA' column: 587

Unique values in 3P%: 214
Number of duplicates in '3P%' column: 465

Unique values in 2P: 83
Number of duplicates in '2P' column: 596

Unique values in 2PA: 134
Number of duplicates in '2PA' column: 545

Unique values in 2P%: 273
Number of duplicates in '2P%' column: 406

Unique values in eFG%: 275
Number of duplicates in 'eFG%' column: 404

Unique values in FT: 67
Number of duplicates in 'FT' column: 612

Unique values in FTA: 80
Number of duplicates in 'FTA' column: 599

Unique values in FT%: 263
Number of duplicates in 'FT%' column: 416

Unique values in ORB: 39
Number of duplicates in 'ORB' column: 640

Unique values in DRB: 79
Number of duplicates in 'DRB' column: 600

Unique values in TRB: 101
Number of duplicates in 'TRB' column: 578

Unique values in AST: 83
Number of duplicates in 'AST' column: 596

Unique values in STL: 22
Number of duplicates in 'STL' column: 657

Unique values in BLK: 22
Number of duplicates in 'BLK' column: 657

Unique values in TOV: 40
Number of duplicates in 'TOV' column: 639

Unique values in PF: 40
Number of duplicates in 'PF' column: 639

Unique values in PTS: 223
Number of duplicates in 'PTS' column: 456
```

Fig. 11. Continuation of results with Fig. 10

The next code shows which columns do not have duplicated values in their rows, thus all the columns contain unique values within each column. This happens if the column's unique count of values equals the total row count in the data set. The execution displays that there are no columns or attributes where all its records are unique, signifying existing duplicate values.

```
unique_columns = []
for column in df.columns:
    if df[column].nunique() == len(df):
        unique_columns.append(column)

print("Columns where each row is unique:")
print(unique_columns)
```

```
Columns where each row is unique:
[]
```

Fig. 12. Output of code for identifying columns where each row is unique.

The researchers then identified all unique values in each attribute. It helps understand the distribution of categorical variables, detect anomalies, and identify potential data inconsistencies. This is very useful for feature selection and data cleaning because it allows you to check for unexpected values or patterns in the dataset.

```
for column in df.columns:
    print(f"Unique values in column '{column}':")
    print(df[column].unique())
    print()
```

```
Unique values in column 'Player':
['Precious Achiuwa' 'Steven Adams' 'Bam Adebayo' 'Ochai Agbaji'
 'Santi Aldama' 'Nickeil Alexander-Walker' 'Grayson Allen' 'Jarrett Allen'
 'Jose Alvarado' 'Kyle Anderson' 'Giannis Antetokounmpo'
 'Thanasis Antetokounmpo' 'Cole Anthony' 'OG Anunoby' 'Ryan Arcidiacono'
 'Deni Avdija' 'Deandre Ayton' 'Udoka Azubuike' 'Marvin Bagley III'
 'Patrick Baldwin Jr.' 'LaMelo Ball' 'Mo Bamba' 'Paolo Banchero'
 'Desmond Bane' 'Dalano Banton' 'Dominick Barlow' 'Harrison Barnes'
 'Scottie Barnes' 'RJ Barrett' 'Will Barton' 'Charles Bassey'
 'Keita Bates-Diop' 'Nicolas Batum' 'Darius Bazley' 'Bradley Beal'
 'Malik Beasley' 'MarJon Beauchamp' 'D?vis Bert?ns' 'Patrick Beverley'
 'Saddiq Bey' 'Khem Birch' 'Goga Bitadze' 'Bismack Biyombo'
 'Buddy Boeheim' 'Bogdan Bogdanovi?' 'Bojan Bogdanovi?' 'Bol Bol'
 'Leandro Bolmaro' 'Devin Booker' 'Brandon Boston Jr.' 'Chris Boucher'
 'James Bouknight' 'Jamaree Bouyea' 'Tony Bradley' 'Malaki Branham'
 'Jarrell Brantley' 'Christian Braun' 'Mikal Bridges' 'Oshae Brissett'
 'Malcolm Brogdon' 'Dillon Brooks' 'Bruce Brown' 'Greg Brown III'
 'Jaylen Brown' 'Kendall Brown' 'Moses Brown' 'Sterling Brown'
 'Troy Brown Jr.' 'Jalen Brunson' 'Thomas Bryant' 'Reggie Bullock'
 'Alec Burks' 'Deonte Burton' 'Jared Butler' 'Jimmy Butler' 'John Butler'
 'Jamal Cain' 'Kentavious Caldwell-Pope' 'Facundo Campazzo'
 'Vlatko ?an?ar' 'Clint Capela' 'Vernon Carey Jr.' 'Jevon Carter'
 'Wendell Carter Jr.' 'Michael Carter-Williams' 'Alex Caruso'
 'Julian Champagnie' 'Justin Champagnie' 'Kennedy Chandler' 'Max Christie'
 'Josh Christopher' 'Brandon Clarke' 'Jordan Clarkson' 'Nic Claxton'
 'Amir Coffey' 'John Collins' 'Zach Collins' 'Chance Comanche'
 'Mike Conley' 'Pat Connaughton' 'Xavier Cooks' 'Robert Covington'
 'Torrey Craig' 'Jae Crowder' 'Jarrett Culver' 'Cade Cunningham'
 'Seth Curry' 'Stephen Curry' 'Dyson Daniels' 'Anthony Davis'
 'Johnny Davis' 'Terence Davis' 'JD Davison' 'Darius Days'
 'Dewayne Dedmon' 'Matthew Dellavedova' 'DeMar DeRozan' 'Moussa Diabaté'
 'Mamadi Diakite' 'Hamidou Diallo' 'Gorgui Dieng' 'Ousmane Dieng'
 'Spencer Dinwiddie' 'Donte DiVincenzo' 'Luka Don?i?' 'Tyler Dorsey'
 'Luguentz Dort' 'Ayo Dosunmu' 'Devon Dotson' 'Jeff Dowtin' 'PJ Dozier'
 'Goran Dragi?' 'Andre Drummond' 'Chris Duarte' 'David Duke Jr.'
 'Kris Dunn' 'Kevin Durant' 'Jalen Duren' 'Tari Eason' 'Anthony Edwards'
 'Kessler Edwards' 'Keon Ellis' 'Joel Embiid' 'Drew Eubanks'
 'Bruno Fernando' 'Dorian Finney-Smith' 'Malachi Flynn'
 'Simone Fontecchio' 'Bryn Forbes' 'Trent Forrest' 'Michael Foster Jr.'
```

Fig. 13. Code execution for displaying all unique values in each attribute

A code was then executed that displays the sum of duplicate player entries. The dataset currently holds 140 players with duplicate records, mostly due to being traded during the NBA season, thus, playing to another team, generating a new record.

```
print("Number of duplicate player entries:", df["Player"].duplicated().sum())
```

```
Number of duplicate player entries: 140
```

Fig. 13. Number of duplicate player entries

To handle the duplicate entries, a line of code was used in order to merge player data and averages as one record, for further optimization of the player's overall statistics. Afterwards, the number of duplicate player entries was checked again after the merging process.

```
df["Original_Order"] = df.index  # Save original order

# Step 3: Group by Player and aggregate stats
df = df.groupby("Player", as_index=False).agg({
    "Age": "first",  # Keep the first recorded age
    "G": "sum",  # Sum of games played
    "GS": "sum",  # Sum of games started
    "MP": "mean",  # Averaging minutes per game
    "FG": "sum", "FGA": "sum", "FG%": "mean",
    "3P": "sum", "3PA": "sum", "3P%": "mean",
    "2P": "sum", "2PA": "sum", "2P%": "mean",
    "eFG%": "mean",
    "FT": "sum", "FTA": "sum", "FT%": "mean",
    "ORB": "sum", "DRB": "sum", "TRB": "sum",
    "AST": "sum", "STL": "sum", "BLK": "sum",
    "TOV": "sum", "PF": "sum", "PTS": "sum",
    "Original_Order": "min"  # Keep the first appearance order
})

# Step 4: Sort back to original order
df = df.sort_values(by="Original_Order").drop(columns=["Original_Order"])

# Step 5: Reset index
df.reset_index(drop=True, inplace=True)
df.head()
```

Fig. 14. Lines of code for players with duplicate entries

```
print("Number of duplicate player entries after merging:", df["Player"].duplicated().sum())
```

```
Number of duplicate player entries after merging: 0
```

Fig. 15. No more duplicate player entries after merging

d) Outlier Detection

Detecting outliers in our dataset is an essential requirement for ensuring that our regression analysis is accurate. Extreme values can potentially pull up or bring down average statistical measures, and model performance will be affected. These outliers might result from data entry errors, exceptional player performances, or very rare gameplay scenarios. If detected and removed, they will maintain the reliability of our models and prevent misleading conclusions (DASCA, 2024) [5].

The researchers first used a code called *detect_outliers_iqr(df)* for detecting numerical outliers for each attribute inside the dataset, which is then visualized through boxplots. The code generated boxplots for each attribute, efficiently detecting outliers such as the boxplot for age in Fig. 16, helping the researcher on what to do next with these outliers.

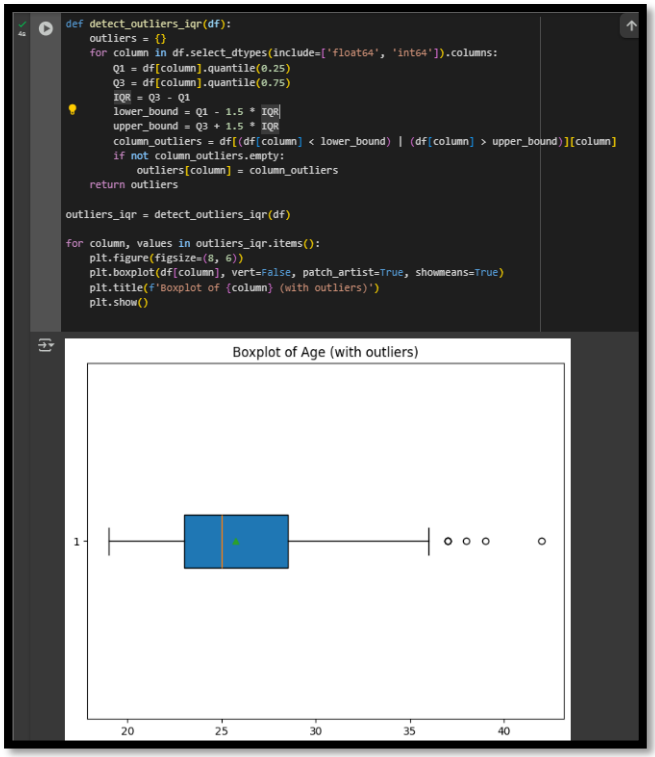


Fig. 16. Outlier detection with sample boxplot

The cleaning then moves forward by removing outliers with the *remove_outliers_iqr(df)* function, filtering out the extreme values, and then visualized again through boxplots, in order to use a more normalized distribution of data.

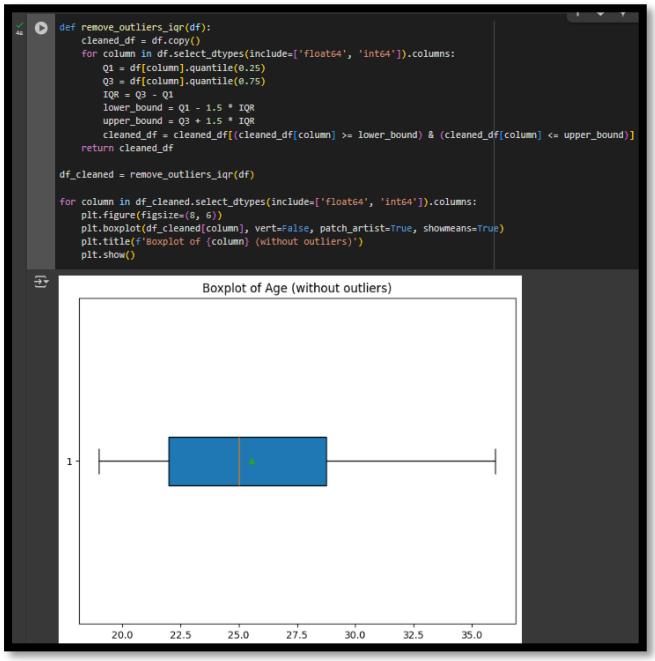


Fig. 17. Visualization of the same boxplots after outlier removal

After the removal of outliers, the shape of the dataset is checked again, now, as its cleaned version that will be used for attribute selection and model building.

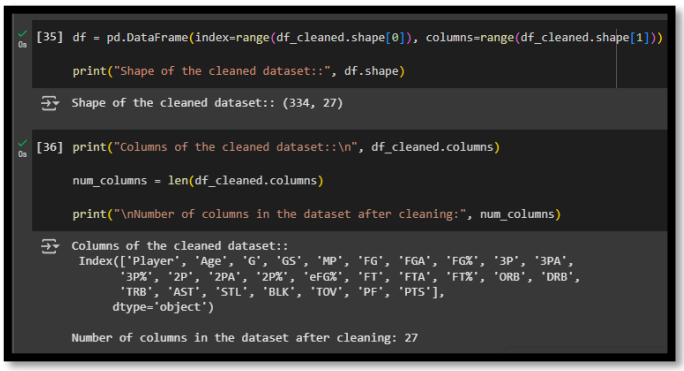


Fig. 18. The current shape, existing columns, and number of columns in the dataset after data cleaning

Finally, histograms were generated to visualize the cleaned dataset, in order to further understand the distribution of numerical attributes.

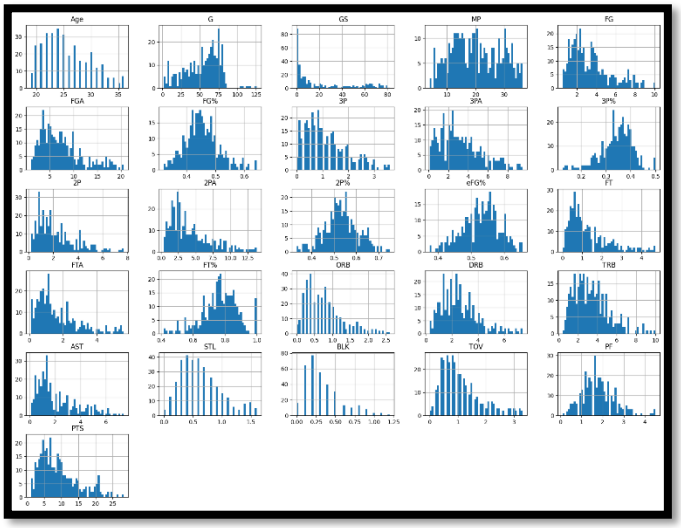


Fig. 19. Histograms of numerical attributes in the cleaned dataset

III. Attribute Selection

Feature selection is one of the primary steps of data preprocessing for machine learning to select the most relevant features and remove redundant and highly correlated attributes. This will improve the model performance, minimize overfitting, and enhance interpretability. Techniques that are used in effective feature selection include statistical tests, domain knowledge-based selection, correlation analysis, feature vectorization, and even dimensionality reduction methods like PCA, to develop a high-quality dataset for modeling. Careful application of these techniques by practitioners should develop more efficient and accurate models for machine learning (Mohamed Habib, J, 2024) [6].

For this study, correlation analysis was used for attribute selection. Correlation analysis is an important part of the study in this regard, which helps understand relationships between numerical attributes. It tells the degree to which two variables are related to each other and thus helps better select predictor attributes. By focusing on strong correlations, we will be able to focus on the most relevant variables, thereby increasing the accuracy of the model and reducing unnecessary complexity. Moreover, correlation helps make predictions because knowledge of one variable can help estimate another. This improves the reliability of data-driven insights, leading to more effective analysis and decision-making (geeksforgeeks, 2024) [7].

a) Correlation Matrix Heatmap

For the study, a heatmap visualization was utilized for providing an overview of the relationships between the attributes in the dataset. It presents a matrix of correlations for all attributes.

Dark red shades correspond to a positive strong correlation of closeness to 1 and dark blue indicates strong negative correlations near -1. Neutral shades of color corresponding closer to the number 0 point to weaker or no connection between variables at all. The values on the diagonal are always 1 because a variable is perfectly correlated with itself. This kind of visualization helps identify highly correlated independent variables that can introduce multicollinearity and redundancy in predictive models. It also picks up features strongly related to the target variable and supports effective feature selection.

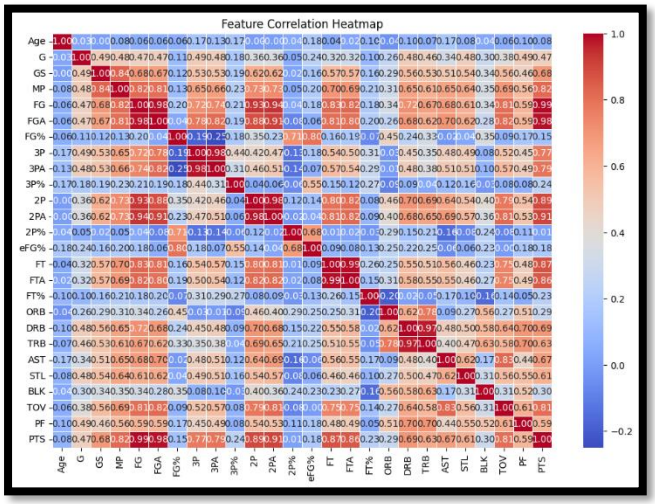


Fig. 20. Correlation Matrix Heatmap

b) Correlation Analysis

After a process of trial and error in selecting a suitable target attribute, the correlation analysis results are as follows, showing how each numerical attribute in the dataset correlates with Field Goal Attempts (FGA). FGA has an obvious perfect correlation of 1.0 with itself. The other highest positive correlations are found for FG (Field Goals per game) and PTS (Points), which both are at 0.98. This means that players who attempt more field goals tend to score more points and convert more shots.

In addition, MPs, 3PAs, and 2PAs present high positive associations with FGA as well as 0.81, 0.82, and 0.91, respectively. This implies players who make many field goal attempts tend to be those playing greater minutes and throwing more shots up from two point and three points. Likewise, FTA and FT also possess correlation magnitudes of 0.80 and 0.81 that are quite substantial, suggesting that players who shoot more field goals tend to commit more fouls and take more free throws as well.

On the other hand, there are very weak correlations between attributes like FG% and 2P% and FGA: 0.04 and -0.08, respectively, which means the frequency of shot attempts does not necessarily indicate shooting efficiency. In this regard, it is also important to make a distinction between volume and efficiency in the assessment of player performance.

Correlation coefficients of each numerical attribute with 'target' attribute:

Age: 0.06411233701966355
G: 0.4709072164079797
GS: 0.6668997899026181
MP: 0.8053665643425372
FG: 0.9820011822313364
FGA: 1.0
FG%: 0.03668329522183154
3P: 0.7811247196027808
3PA: 0.8181766718602751
3P%: 0.18636474202710873
2P: 0.8789574478081833
2PA: 0.9114231921192107
2P%: -0.08047889506020736
eFG%: 0.057698403523500696
FT: 0.8085753249815283
FTA: 0.8034912927628584
FT%: 0.20105649258432073
ORB: 0.26346100480766765
DRB: 0.6848340938468298
TRB: 0.6170129534967008
AST: 0.7003298184219823
STL: 0.62495215000699
BLK: 0.2844348573151982
TOV: 0.8153656932133828
PF: 0.5887689844606658
PTS: 0.9834709892291885

Fig. 21. Correlation coefficients with 'FGA' as target attribute

The figure above (Fig.21) displays the correlation coefficients with 'FGA' as target attribute, with highlighted attributes signifying that these attributes have high correlation with the target attribute.

c) Correlation Visualization

The program also allows to interactively choose a predictor attribute and a target attribute from the cleaned dataset, calculate the correlation coefficient (r) for how strong and in which direction the relation is, and generate a scatter plot that represents the two attributes' relationship visually. This approach is useful in feature selection as it aids in the identification of those features that have a large impact on the target variable.

Correlation analysis helps determine which attributes are most relevant to Field Goal Attempts (FGA). Highly correlated attributes can be reduced, and model efficiency improved, while ensuring that the most influential features are used for further analysis or predictive modeling.

```
Enter the predictor attribute: PTS
Enter the target attribute: FGA

Correlation coefficient (r) between PTS and
FGA: 0.9834709892291885
```

Fig. 22. Input and Output generated for finding correlation coefficient

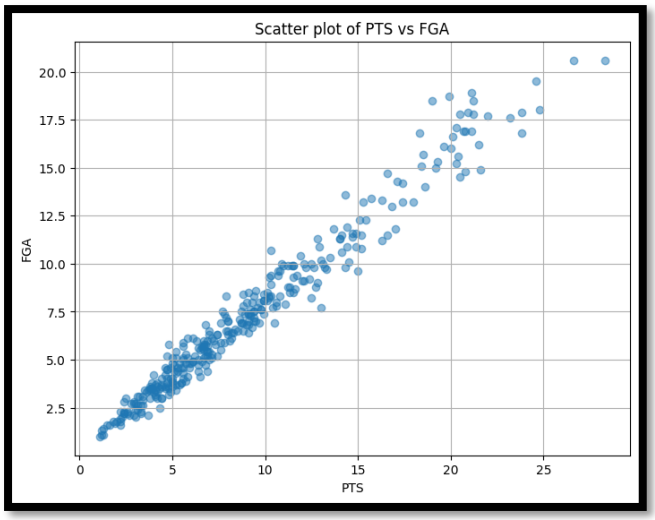


Fig. 23. Scatter Plot of correlation of Field Goals Attempted per game (FGA) and Points made (PTS)

The correlation coefficient, r, for PTS and FGA is 0.9835, almost exactly 1. This implies a very high positive correlation, meaning that as the field goal attempts of a players go up, the points scored rise in association with it.

IV. Model Building

a) Simple Linear Regression

From the provided source code, the first section of the code reads in the data and necessary libraries for creating a simple linear regression model. It imports the necessary modules such as LinearRegression from sklearn.linear_model, train_test_split from sklearn.model_selection, and matplotlib.pyplot to visualize the data. Then, it checks whether predictor and target attributes have been selected by verifying if they are present in the dictionary selected_attributes. If either of the attributes is missing, a ValueError is raised asking for their specification before proceeding. Once confirmed, the selected attribute names are retrieved and stored in attr1 and attr2. Then, the predictor variable, X, is defined by extracting the selected column from df_cleaned as a DataFrame, and the target variable, y, is stored. This step ensures that the data is well-structured before training the regression model.

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np

if "attr1" not in selected_attributes or "attr2" not in selected_attributes:
    raise ValueError("Attributes not set. Please run the attribute selection cell first.")

attr1 = selected_attributes["attr1"]
attr2 = selected_attributes["attr2"]


X = df_cleaned[[attr1]]
y = df_cleaned[attr2]

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

Fig. 24. Preparation and initialization of data for simple linear regression use

The latter part of the code trains a basic linear regression model using the chosen predictor and target variables. Performance is evaluated using an R-squared (R^2) score; the higher it is, close to 1, the more strongly related are the variables involved. This will print the R^2 scores of both the training and test set to evaluate the generalization power of the model. The scatter plot of the prediction of the model is done through a scatter plot of the training data and the fitted regression line, which can be used to see how good the model represents the relationship.

```python
linreg = LinearRegression().fit(X_train, y_train)

print(f'Linear model intercept: {linreg.intercept_}')
print(f'Linear model coefficient: {linreg.coef_[0]}')
print(f'R-squared score (training): {linreg.score(X_train, y_train):.3f}')
print(f'R-squared score (test): {linreg.score(X_test, y_test):.3f}')

plt.figure(figsize=(5, 4))
plt.scatter(X_train, y_train, marker='o', label="Training Data")
plt.plot(X_train, linreg.predict(X_train), 'r-', label="Regression Line")
plt.title(f'Least-Squares Linear Regression: {attr1} vs {attr2}')
plt.xlabel(attr1)
plt.ylabel(attr2)
plt.legend()
plt.show()
```

Fig. 25. Simple Linear Regression Model Coefficients and Performance Metrics

```
Linear model intercept: 0.29298287079187624
Linear model coefficient: 0.7776438368834887
R-squared score (training): 0.967
R-squared score (test): 0.969
```

Fig. 26. Regression Coefficients and Model Performance Metrics

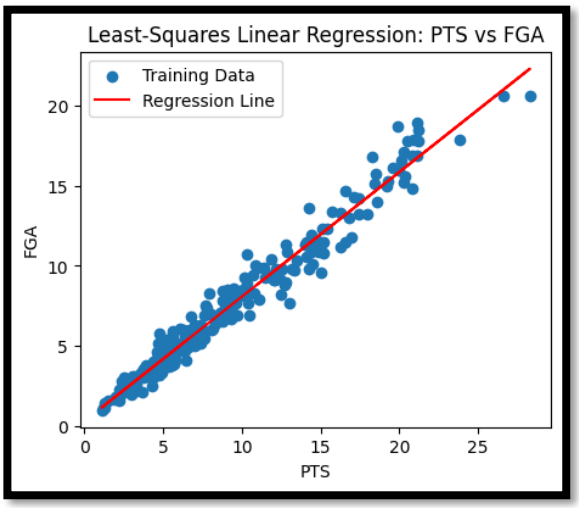


Fig. 27. Scatter Plot with Regression Line for PTS vs. FGA

The simple linear regression model is helpful in generating key results about the relationship between selected predictor and target variables. The intercept of the linear model is 0.293, meaning that if the predictor variable is zero, then the target variable would be at the expected value. A linear model coefficient of 0.778 reflects the change of about 0.778 units for the increase of one unit of the predictor variable in the target variable.

The R-squared (R^2) measures how well the model explains the variation in the target variable. The R-squared score for the training set is 0.967 and for the test set is 0.969. Such high values demonstrate the presence of a strong linear relationship between the predictor and target variables, that is, the model can indeed predict the target variable based upon the predictor. The scatterplot with the regression line visually confirms the relationship. This scatterplot shows how close the data points are following the fitted line, giving evidence to reinforce this model.

b) Simple Linear Regression- based prediction of FGA

Finally, this snippet of code enables prediction of the target variable given a new input value for the predictor variable. The user is able to input a new value for PTS, and then converted to a float. Afterwards, it is reshaped into a format that will suit for prediction. For prediction, the trained simple linear regression model computes the corresponding value for FGA from the relationship determined.

```
new_value = float(input(f"Enter a new value for {attr1}: "))
new_value_reshaped = np.array([[new_value]])

predicted_target = linreg.predict(new_value_reshaped)
print(f"Model prediction for {attr2}: {predicted_target[0]:.3f}\n")

Enter a new value for PTS: 20
Model prediction for FGA: 15.846
```

Fig. 28. Output of the model-based prediction code

In this case, when the PTS are 20, the model predicts that the FGA is 15.846, or if a player scores 20 points, they are supposed to attempt approximately 15 field goals. This is making the prediction based on the learned linear relation from the dataset, further establishing how the model could be used for estimating future outcomes based on past trends.

c) Multiple Linear Regression

The Multiple Linear Regression model makes use of a similar library to the Simple Linear Regression model. It takes the list of column names from df_cleaned and stores it in predictors, and sets FGA as the target variable. It then computes the correlation matrix of numeric columns to help choose the most relevant predictor variables. The user is asked to input the top predictor attributes in a comma-separated list and this is automatically translated into a list of selected predictors. This is followed by the splitting of the dataset into 80% training and 20% testing, ensuring that the model has been trained on separate data than which it would be tested and assessed.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

predictors = list(df_cleaned.columns)
target = 'FGA'

corr_matrix = df_cleaned.select_dtypes(include=['float64', 'int64']).corr()
print(corr_matrix)

selected_predictors = input("Enter the top predictor attributes to use (comma-separated): ").split(",")

X = df_cleaned[selected_predictors]
y = df_cleaned[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Fig. 28. Preparation and initialization of data for multiple linear regression use

The intercept is the value of the target variable, FGA, at which all the predictor variables are zero. Coefficients for each selected predictor indicate how much the target variable is expected to change for a one-unit increase in each predictor. R-squared scores measure the quality of fit of a model toward strength of fit in both the training and testing sets. Values that are closer to 1 will indicate a better fit. Having a high train score means your model is well fitting the data in the train set, whereas having similar train and test scores maximizes your confidence that it generalizes correctly to new data. If the testing score is significantly smaller, then it means there is probably overfitting, where the model captures more noise rather than the relationship in question.

```
linreg = LinearRegression()
linreg.fit(X_train, y_train)

print(f"Intercept: {linreg.intercept_}\n")
print("Coefficients:")
for i, col in enumerate(selected_predictors):
    print(f"{col}: {linreg.coef_[i]}")

print(f"\nTraining R-squared score: {linreg.score(X_train, y_train)}")
print(f"Testing R-squared score: {linreg.score(X_test, y_test)}")
```

Fig. 29. Multiple Linear Regression Model Coefficients and Performance Metrics

```
Enter the top predictor attributes to use
(comma-separated): PTS,2PA,FG

Intercept: 0.2773834561787849

Coefficients:
PTS: 0.48666052856181036
2PA: 0.06878715396536123
FG: 0.7134238670552123

Training R-squared score:
0.9708302092701263
Testing R-squared score: 0.968414142423373
```

Fig. 30. Multiple Linear Regression Prediction Output

The multiple linear regression model predicts FGA based on three key predictor variables: PTS, 2PA, and FG. The R^2 value on the training data was 0.9708 and on the testing data was 0.9684. This implies that nearly 97% of the variance in FGA is explained by the selected predictors, and a minimal gap between training and testing scores indicates a good generalization of the model to new data.

Each coefficient represents the change in FGA for a one-unit increase in the respective predictor, assuming that all other variables are held constant. Points Scored (PTS) has the largest effect of 0.4867. This means that for every additional point scored, FGA increases about 0.49 attempts per extra point. The Field Goals Made (FG) coefficient of 0.7134 implies that for each shot made, there are about 0.71 shots taken, which is intuitive as making the shot usually requires the act of taking the shot. Lastly, Two-Point Attempts (2PA) has a smaller coefficient of 0.0688, which indicates a weaker but still positive relationship with FGA.

Overall, the results support the premise that the more effective players take more shots, and the robust explanatory power of the model indicates that those three variables are good predictors of shot attempts.

V. DISCUSSION

a) Model Performance Evaluation of both models

Primary differences between simple and multiple linear regression models are complexity and strength in predictive capability. Simple linear regression model utilized the PTS as a predictor variable to approximate FGA, which resulted in a very high R^2 score of 0.967 on the training data and an R^2 score of 0.969 on test data. It indicates a strong relationship that holds between PTS and FGA: the points scored, being a good predictor of attempts made by a player for field goal attempts. This could, however overlook other factors influencing FGA that are also of importance, for example, efficiency, playing time, or turnover rates.

The multiple linear regression model used three predictor variables: PTS, 2PA, and FG. It, obtained somewhat higher training R^2 at 0.9708 and testing R^2 at 0.9684. This means adding more variables allows for a sense of how separate factors help explain FGA more concretely. This means that from the coefficients, the highest coefficient of influence for FGA is PTS, followed by FG and 2PA and still reinforces that scoring and shot efficiency are amongst major drivers of shot attempts. The similarity of both training and testing R^2 scores in the two models suggests that neither model is overfitting the data significantly.

Multiple regression offers more explanation and a closer description of relationships between FGA and other metrics related to the performance. Such models are even helpful in counterbalancing possible effects of omitted variable bias as seen when applying a simple model fails to represent major influential variables. Nevertheless, in ensuring there is no overly complicated model to explain, additional predictors could raise issues with multicollinearity that can impair clarity of a model's output.

b) Reflection on Attribute Selection and Its Impact on Model Performance

The selection of predictor variables is critical in the construction of an efficient regression model. The attribute selection for the multiple regression model was based on a correlation matrix that identified features with strong correlations with FGA. Highly correlated attributes such as PTS, FG, and 2PA were added to the model, which showed high R^2 scores, meaning that the model was able to predict well. Furthermore, the narrow gap between training and testing scores suggests that the chosen features generalize well to new data; hence, the possibility of overfitting is minimized.

A critical insight from the attribute selection process is that although PTS was already a very strong predictor in the simple regression model, adding FG and 2PA added context to shooting efficiency and attempt frequency. Adding too many predictors, however, could lead to redundancy and increased complexity without substantial gains in predictive power. For example, adding MP (Minutes Played) and TOV (Turnovers) in another model version slightly increased R^2 but perhaps introduced unnecessary complexity because these variables have a more indirect effect on FGA.

The choice of attributes puts a balance between predictive accuracy, generalization, and interpretability. Selecting the most relevant variables rather than stacking more ensures that the model stays both statistically sound and practically useful for analysis and decision-making.

VI. CONCLUSION AND REFLECTION

a) Conclusion

This performance task showed how simple and multiple linear regression can be used to predict FGA based on different performance metrics obtained during the play of a player. The simple regression model with PTS as only the predictor variable achieved an impressive R^2 score of 0.967 on training data and also 0.969 on test data, signifying that scoring and shot attempts tend to be highly related. But the multiple regression, which included PTS, 2PA (Two-Point Attempts), and FG (Field Goals Made), gave slightly better performance with R^2 scores of .9708 on training data and 0.9684 on testing data, which shows that more predictors are considered to provide a more holistic view of the factors behind FGA.

The results indicate that the most important effect was for PTS on FGA, followed by FG and 2PA. The close scores of training and testing for both models indicated good generalization ability and avoided overfitting. More generally, the results further confirm the assumption that although a single predictor can give informative results, multiple related features combined give much better accuracy and robustness to the prediction model.

b) Reflection

This example of linear regression on actual basketball data really helped me practice how to prepare data, choose models, and assess their performance. The step of selecting predictors from correlation analysis is a key reminder of feature selection in model building [7]. It is also very enlightening to see how various predictors contribute to model accuracy and that there is always a trade-off between the simplicity of a model and its power to predict [6].

c) Challenges

Selecting the most appropriate features was one of the biggest issues I faced with this task. including too many predictors risked redundancy and multicollinearity, which distorted the reliability of the model. Multiple regression also became more difficult to interpret as compared to simple regression, because every coefficient required an analysis within the context of others. Techniques such as standardization of coefficients and visualization make the interpretation clear. Balancing complexity with performance was very essential to keep it generalizing very well with the model to not overfit.

VII. REFERENCES

[1] Vinco, V. (2023). 2022-2023 NBA Player Stats. https://www.kaggle.com/datasets/vivovinco/20222023-nba-player-stats-regular/data?select=2022-2023+NBA+Player+Stats+-+Regular.csv

[2] Geeksforgeeks (2024). How to Perform Feature Selection for Regression Data. https://www.geeksforgeeks.org/how-to-perform-feature-selection-for-regression-data/

[3] Nasima (2024). Effective Strategies for Handling Missing Values in Data Analysis. https://www.analyticsvidhya.com/blog/2021/10/handling-missing-value/#What_Is_a_Missing_Value?

[4] End, N. (2023). What is Data Cleaning? https://rowzero.io/blog/data-cleaning

[5] Data Science Council of America (DASCA) (2024). Why Detecting Outliers is Crucial for Accurate Data Analysis? https://www.dasca.org/world-of-data-science/article/why-detecting-outliers-is-crucial-for-accurate-data-analysis#:~:text=Given%20their%20potential%20to%20skew,visualization%20and%20statistical%20techniques%20include%3A&text=The%20Z-score%20method%20detects,in%20terms%20of%20standard%20deviations.

[6] Mohamed Habib, J (2024). Feature Selection Techniques in Machine Learning. https://medium.com/%40jaberi.mohamedhabib/feature-selection-techniques-in-machine-learning-4c395e4a8205

[7] Geeksforgeeks (2024). Correlation: Meaning, Significance, Types and Degree of Correlation. https://www.geeksforgeeks.org/correlation-meaning-significance-types-and-degree-of-correlation/#significance-of-correlation